Grounded Cache Routing for Retrieval-Augmented Generation: When Is It Safe to Reuse an Answer?

Syed Huma Shah
syedhuma.shah@duke.edu

May 21, 2026

Abstract

Modern retrieval-augmented generation (RAG) deployments increasingly rely on caching to reduce token cost and time-to-first-token (TTFT). Prefix-level KV reuse is now standard in serving stacks such as vLLM, and chunk-level and position-independent reuse have been pushed further by recent systems (RAGCache, TurboRAG, CacheBlend, EPIC, ContextPilot, PCR, LMCache). Output-level *semantic answer caches*, by contrast, remain fragile: similar prompts can map to different correct answers, retrieved evidence drifts as the corpus is updated, and adversarial collision attacks have been shown to hijack cached responses. We argue that the right framing for cached answer reuse is not *how to reuse faster* but *when reuse is safe*. We propose GroundedCache, an evidence-validated cache router that admits a cached answer only when four cheap gates simultaneously hold: query similarity, retrieved-evidence overlap, source-version validity, and lexical (or judge-based) support of the cached answer by the freshly retrieved evidence. We build a six-regime workload (exact-repeat, paraphrase, near-miss, document-drift, long shared-document, bounded-KB CAG) that explicitly stress-tests cache *safety* rather than only hit rate, and an operator-facing metric, the *unsafe-served rate* (USR), defined as the fraction of all queries that received a wrong cached answer, alongside TTFT and token cost. Across two datasets and 12,000 real-LLM generations (Qwen2.5-7B-Instruct served by vLLM with Automatic Prefix Caching), GroundedCache drives USR to 0.0% on every HotpotQA regime (vs. 15–35% under naive semantic caching) and to 1.5% on mtRAG document drift (vs. 51.5% under naive), a 34× reduction in wrong cached answers per query on the system's design-point adversarial regime, and 3–10× reductions across the other mtRAG regimes. End-to-end p50 latency under GroundedCache stays within 1.04–1.07× of a no-cache RAG baseline; the no-support variant trades USR for 1.4–1.5× speedup. A per-gate marginal ablation isolates the lexical support gate as the load-bearing safety mechanism on both datasets (+0.125 USR on HotpotQA, +0.118 on mtRAG when removed), with the remaining gates providing defense-in-depth at near-zero cost. We release the end-to-end implementation, traffic-synthesis tooling, and evaluation harness so the framework can be ported to richer datasets and stronger compressors.

1 Introduction

A typical RAG pipeline repeatedly does the same expensive work for queries that are functionally indistinguishable to the user: it embeds the query, runs nearest-neighbor search over a corpus, concatenates the top-k chunks into a prompt, and decodes a long-context answer. Three layers of caching can in principle eliminate that work:

1. *Prefix / KV cache reuse* at the serving layer (vLLM APC [1], SGLang RadixAttention [2]), and its RAG-aware extensions (RAGCache [3], TurboRAG [4], CacheBlend [5], EPIC [6], ContextPilot [7], PCR [8], LMCache [9]).

2. *Retrieval-result reuse*, where the top-k document set itself is cached for similar queries (Proximity [10], QVCache [11]).

3. *Output / answer reuse*, where the entire generated answer is returned for a semantically similar prior query (GPTCache [12], ContextCache [13]).

The first two have strong correctness guarantees by construction: KV reuse preserves token outputs exactly when prefixes match, and retrieval reuse is safe as long as the underlying corpus is stable. Output-level semantic caching does not. A cosine-similar prior query can map to a cached answer whose underlying evidence is no longer valid, no longer the same evidence, or never supported the question in the first place. Recent work has shown that naive semantic caches are even adversarially exploitable, with reported 86% response-hijacking rates in evaluated scenarios [14].

We make three contributions:

1. *An evidence-validated cache router (*GroundedCache*).* A cached answer is admitted only when (G1) the new query is semantically close to the cached query, (G2) the new retrieved evidence shares a high Jaccard with the cached evidence signature, (G3) shared chunks carry the same source version, and (G4) the cached answer's content tokens are covered by the new evidence (or, optionally, verified by a lightweight judge call). The router falls back to query-conditioned compression and generation when any gate fails.

2. *A six-regime workload for cache-safety stress testing.* We provide deterministic synthesis for exact-repeat, paraphrase, near-miss, document-drift, long shared-document, and bounded-KB CAG traffic over arbitrary RAG corpora, plus a HotpotQA adapter. The document-drift regime mutates numeric tokens in the gold documents so that a cached answer becomes *verifiably* wrong unless rejected.

3. *Operator-facing metric stack.* Beyond TTFT, latency, tokens, and answer-cache hit rate we report the *unsafe-served rate* USR (fraction of all queries that received a wrong cached answer), the conditional false-hit rate FH (per served cached answer), and stale-/unsupported-hit decompositions, so the safety/reuse trade-off is auditable at both the system and the per-query level.

Our framing is complementary to, rather than competitive with, prefix-level work: GroundedCache treats vLLM APC and chunk-level KV reuse as orthogonal serving primitives operating below the router. The contribution is a *policy layer* on top.

2 Related Work

Prefix and KV caching are now standard in open-source serving stacks. vLLM's Automatic Prefix Caching [1] and SGLang's RadixAttention [2] reuse KV blocks when prompt prefixes align. The line of work on RAG-specific KV reuse moves beyond exact prefix matching: RAGCache [3] caches intermediate states of retrieved documents and reports up to 4× TTFT and 2.1× throughput gains; TurboRAG [4] pre-computes document KV caches offline for up to 9.4× TTFT reduction; CacheBlend [5] corrects reused non-prefix chunks via selective recomputation. EPIC [6] formalizes position-independent caching, and ContextPilot [7] adds context indexing, ordering, and de-duplication. PCR [8] adds prefetch-enhanced reuse, and LMCache [9] provides multi-tier KV storage and cross-engine reuse. All of these target the *how* of reuse and largely assume the *when* is safe.

Semantic answer caching has a separate failure-mode literature. GPTCache [12] is the canonical open-source baseline. ContextCache [13] extends the cache key with multi-turn dialogue context, demonstrating that turn-level similarity is insufficient when the referent has shifted. Proximity [10] caches retrieval results rather than outputs, validating that retrieval-side caching is safer. QV-Cache [11] learns region-specific thresholds for safe ANN-level reuse, cutting query latency 40–1000$\times$ on hits without compromising recall. Recent work shows naive semantic caching is vulnerable to key-collision attacks with up to 86% response-hijacking rate [14], and agentic-plan caching suffers high false positives when outputs depend on external state. These three failure modes, referent shift, retrieval drift, and adversarial collision, motivate the four gates in our validator.

The compression literature has moved from hard pruning (RECOMP [15], LongLLMLingua [16], Provence [17], PISCO [18]) to adaptive, query-conditioned, or representation-level compression (OSCAR [19], SeleCom [20], REFRAG [21]). We use a sentence-level selector as a practical baseline; learned compressors plug into the same `compress(query, chunks)` interface.

RAG evaluation frameworks have multiplied in the last two years. RAGAS [22] and ARES [23] provide reference-free and trained-judge frameworks; RAGTruth [24] ships nearly 18K manually annotated outputs for hallucination analysis; RAGBench [25] and RAGChecker [26] give 100K-scale explainable examples and fine-grained diagnostics; mtRAG [27] covers multi-turn settings; T^2-RAGBench [28] adds tabular reasoning. None of these were built specifically for cache-safety evaluation under repeated paraphrases and corpus drift, which motivates our workload contribution.

3 Method

3.1 Pipeline

Figure 1 sketches the end-to-end pipeline; the rest of this section spells out each block.

A query q enters the router. We embed it (the same model used for the retriever, so the embedding can be reused), look up an exact retrieval-cache entry, and on miss fall back to approximate retrieval-cache lookup with cosine threshold τ_{ret}. On total miss we hit the FAISS index for the top-k chunks. We deduplicate chunks by SHA-1 hash and order them by $(\text{doc\_id}, \text{chunk\_id})$ so identical evidence sets produce identical prompt prefixes, which is the precondition for prefix-level KV reuse below.

3.2 Evidence signature

For every served query we record an *evidence signature*:

$$\sigma(q) \stackrel{\text{def}}{=} \big\langle \text{doc\_ids}, \text{chunk\_ids}, \text{chunk\_hashes}, \text{versions}, \text{scores} \big\rangle. \tag{1}$$

The signature is content-addressed (SHA-1 over normalized chunk text) and version-tagged. It serves as the cache key's evidentiary half: similarity between *queries* is necessary for reuse, but agreement of *evidence* is what makes reuse safe.

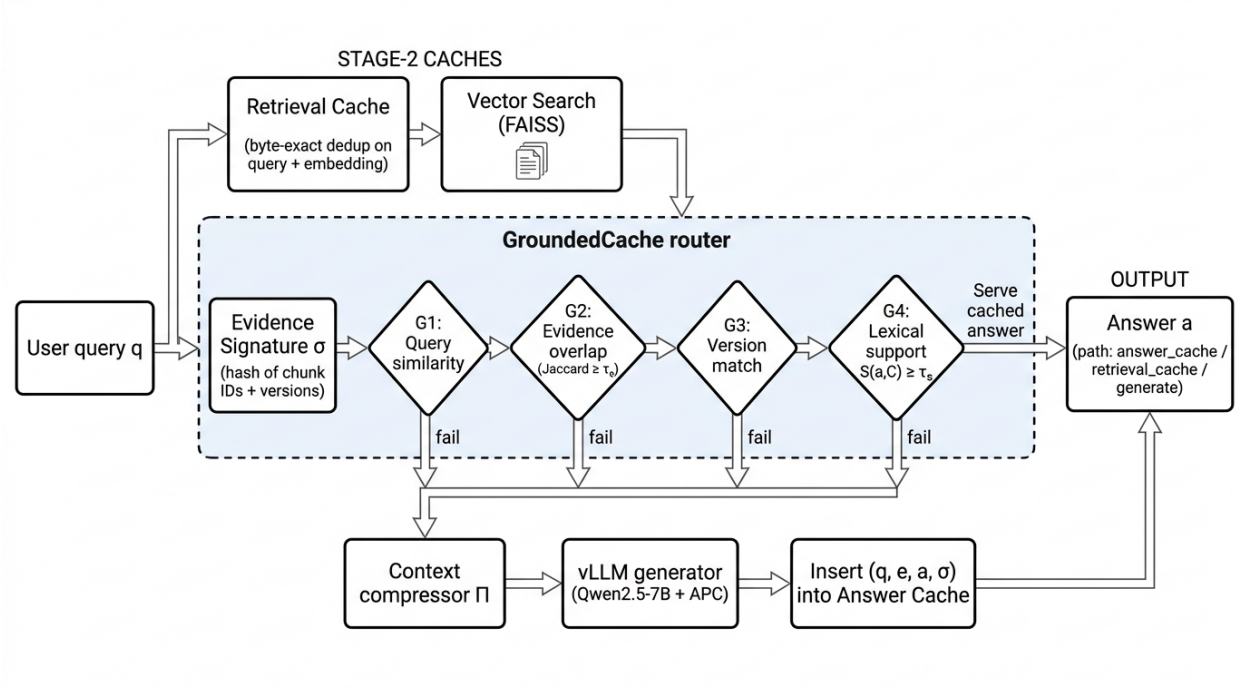

Figure 1: GroundedCache system architecture. A query first hits the stage-2 retrieval cache (byte-exact dedup); on a miss it falls back to FAISS vector search. The retrieved chunks are hashed into an evidence signature σ, and four gates (G1 query similarity, G2 evidence overlap, G3 version match, G4 lexical support) decide whether to serve a cached answer from the answer cache or fall back to the compression + vLLM generate path. The dashed block is the only new component; everything else is unchanged from a standard RAG stack.

3.3 Validation gates

Let (q^c, a^c, σ^c) be a cached entry and (q, σ, C) be the fresh query, fresh signature, and fresh chunk set. We admit a^c iff all four gates fire:

$$\text{(G1) query similarity} \quad \cos(\text{emb}(q), \text{emb}(q^c)) \geq \tau_q \tag{2}$$

$$\text{(G2) evidence overlap} \quad J(\sigma, \sigma^c) \geq \tau_e \tag{3}$$

$$\text{(G3) version match} \quad \text{ver}(c, \sigma) = \text{ver}(c, \sigma^c)\ \forall c \in \sigma \cap \sigma^c \tag{4}$$

$$\text{(G4) evidence support} \quad S(a^c, C) \geq \tau_s \tag{5}$$

where $J(\cdot, \cdot)$ is Jaccard over chunk hashes and $S(a, C)$ is either a deterministic lexical-overlap score (the default; the fraction of content tokens in a that also appear in C) or, optionally, a single-call judge LLM that returns yes/no for "is a supported by C." Algorithm 1 states the full procedure; Figure 2 shows the same logic as a per-query decision flow.

3.4 Compression fallback

When any gate fails we fall back to query-conditioned compression in the spirit of Provence [17] and PISCO [18]: embed each sentence in the retrieved chunks, score against the query embedding, and keep the top sentences subject to a token budget, with a min-one-sentence-per-chunk guar-

Algorithm 1: GROUNDEDCACHE routing for a query q

Input: query q; retriever R; retrieval-cache $\mathcal{R}$; answer-cache $\mathcal{A}$; validator gates with thresholds τ_q, τ_e, τ_s; compressor Π; generator G.

Output: answer a, path label.

$e \leftarrow \text{emb}(q)$; `// single encoder call, reused below`
$C \leftarrow \mathcal{R}.\text{lookup}(q, e)$; `// stage 2 cache`
if $C = \perp$ **then**
  $C \leftarrow R.\text{search}(q, k)$; $\mathcal{R}.\text{insert}(q, e, C)$
end
$C \leftarrow \text{dedup\_and\_order}(C)$; $\sigma \leftarrow \text{signature}(C)$;
$h \leftarrow \mathcal{A}.\text{nearest}(e)$; `// stage 3 lookup`
if $h \neq \perp$ **then**
  if $\cos(e, h.e^c) \geq \tau_q$ ***and*** $J(\sigma, h.\sigma^c) \geq \tau_e$ ***and*** $\text{versions\_match}(\sigma, h.\sigma^c)$ ***and*** $S(h.a^c, C) \geq \tau_s$ **then**
    return $h.a^c$, `answer_cache`
  end
end
$C' \leftarrow \Pi.\text{compress}(q, C)$; `// stage-4 compression fallback`
$a \leftarrow G.\text{generate}(\text{prompt}(q, C'))$;
$\mathcal{A}.\text{insert}(q, e, a, \sigma)$;
return a, `generate`

antee to avoid emptying any retrieved chunk. Learned compressors (OSCAR [19], SeleCom [20], REFRAG [21]) implement the same interface and can be swapped in.

4 Experimental Setup

4.1 Workload regimes

We synthesize six regimes deterministically over any RAG corpus:

exact-repeat re-issue a sampled prior question verbatim.

paraphrase re-issue under a deterministic surface paraphrase.

near-miss re-issue a lexically-similar question whose gold documents are *disjoint* from the lexically-nearest prior question. A semantic cache should reject these; GROUNDEDCACHE should reject them via G2.

document-drift re-issue a prior question after applying a seeded mutation to the numeric tokens of its gold document(s). A version-aware cache should reject these via G3.

long shared-document many questions hitting the same document, the favorable regime for both dedup and prefix-level reuse.

bounded-KB CAG all relevant material assumed in-context; the regime under which cache-augmented generation may beat classical RAG [29].

The mixture is designed so cumulative cache safety can be evaluated, not just hit rate on a benign trace.

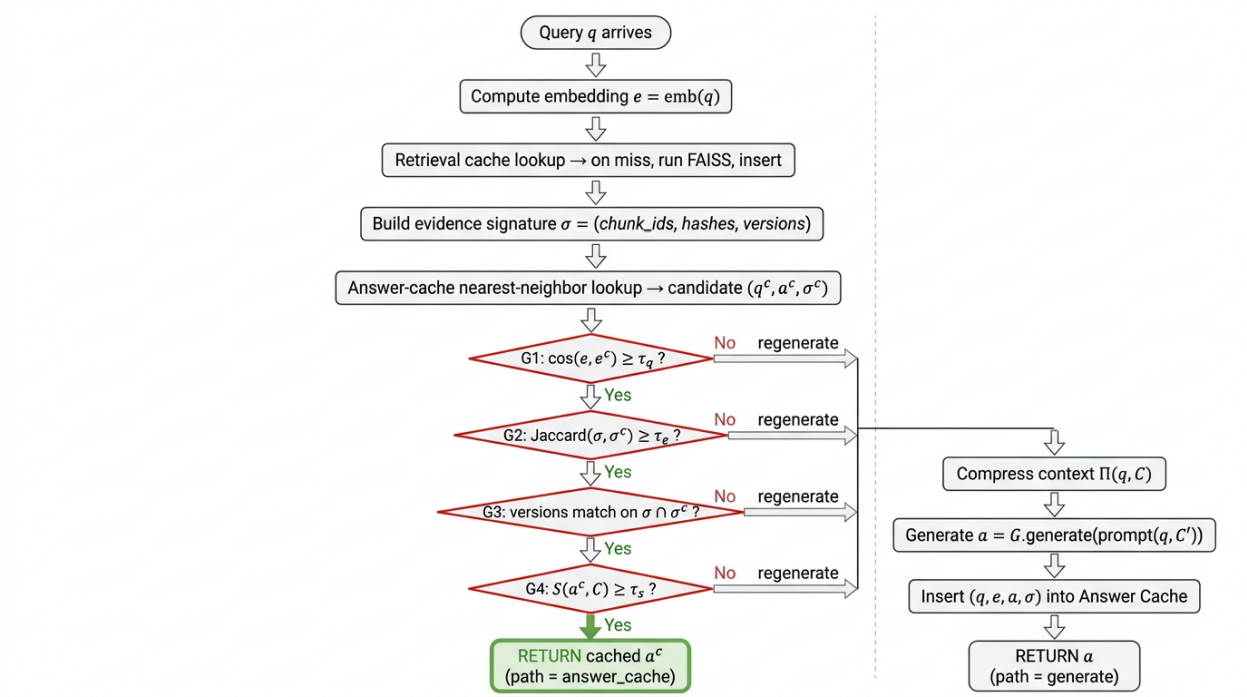

Figure 2: Per-query decision flow inside the router. The four gates (G1–G4) form a short-circuiting chain: any `No` routes the query to the regeneration fallback (compress, generate, insert), preserving safety at the cost of one cache miss. Only when all four fire is the cached answer a^c served (green path). This figure is the visual companion to Algorithm 1.

4.2 Datasets and models

We report results on a tiny hand-crafted 3-document corpus (used as a smoke control) and on the HotpotQA distractor split (each example ships its own context paragraphs). The retriever uses `all-MiniLM-L6-v2` [30] with FAISS-CPU [31]. The generator is an interchangeable adapter: an Anthropic API client, an OpenAI-compatible vLLM endpoint, or a deterministic *extractive* backend that returns the sentence in the retrieved context with the highest query-token overlap. The extractive backend lets us produce reproducible results without an API key while still exercising every validator gate meaningfully (its answers come from evidence, so the support gate S produces non-trivial scores).

4.3 Variants

We compare five router configurations:

- `naive`: semantic answer cache only; all gates relaxed (the stage-3 baseline modeled on GPTCache).
- `no-version`, `no-evidence`, `no-support`: ablations removing one gate at a time.
- `full`: GroundedCache, all four gates active.

4.4 Metrics

For each query we record TTFT, retrieval latency, end-to-end latency, prompt and completion tokens, path (`generate`, `retrieval_cache`, `answer_cache`), exact-match and token-F1 against gold (when available), and substring-containment of gold in the answer (robust to verbose answers). A query is flagged as a *disagreement* when F1 < 0.5 *and* the normalized gold string fails to appear as a substring of the answer; this conjunction prevents F1's punishment of correct paraphrases from inflating safety counters. Let $\not\Vdash_i^{\text{ac}}$ indicate that query i was served from the answer cache and $\not\Vdash_i^{\text{dis}}$ indicate that its answer disagrees with gold. We aggregate:

$$\text{aHR} \stackrel{\text{def}}{=} \frac{1}{N}\sum_{i=1}^{N} \not\Vdash_i^{\text{ac}} \qquad \text{(answer-cache hit rate)} \tag{6}$$

$$\text{USR} \stackrel{\text{def}}{=} \frac{1}{N}\sum_{i=1}^{N} \not\Vdash_i^{\text{ac}} \cdot \not\Vdash_i^{\text{dis}} \qquad \text{(unsafe-served rate)} \tag{7}$$

$$\text{FH} \stackrel{\text{def}}{=} \frac{\text{USR}}{\text{aHR}} = \Pr[\text{dis} \mid \text{ac}] \qquad \text{(conditional false-hit rate)} \tag{8}$$

The lexical support score used by gate G4 (Eq. 5) is

$$S(a, C) \stackrel{\text{def}}{=} \frac{|\,\tau(a) \cap \tau(C)\,|}{|\,\tau(a)\,|}, \quad \tau(x) = \{t \in \text{toks}(x) : t \notin \mathcal{W} \wedge |t| \geq 3\}, \tag{9}$$

where τ extracts content tokens after removing a small stop-word set $\mathcal{W}$, and a cached answer a is rejected when $S(a, C) < \tau_s$ (default τ_s=0.6). USR is our primary safety metric: it directly measures the fraction of all queries that received a wrong cached answer, which is the only number a production operator can act on. We report aHR to capture the trade-off between reuse and safety, and the conditional FH so the table aligns with prior caching evaluations that report FH conditional on hit. Stale-hit (SH) and unsupported-hit (UH) counters are computed the same way (numerator substituted accordingly) and reported in the appendix; we omit them from the main tables because under the full system they are identically zero. Crucially, the retrieval-cache path is excluded from the USR denominator's numerator entirely: a retrieval-cache hit only skips vector search and still regenerates, so by construction it cannot return a stale answer.

5 Results

We report two sweeps with the same router configuration: a multi-hop sweep on HotpotQA (§5.1) and a multi-turn sweep on mtRAG (§5.2). Both use `Qwen/Qwen2.5-7B-Instruct` served by vLLM with Automatic Prefix Caching enabled, behind our router. Each cell sweeps N=200 queries; with six regimes and five router variants this is $6{\times}5{\times}200 = 6000$ generations per dataset. USR is reported over the full N=200 denominator (the operator metric); the conditional false-hit rate FH is reported alongside but its denominator is the answer-cache hit count, which can be small under GroundedCache. For $0/k$ outcomes we report FH$= 0$ but note the Wilson 95%-CI upper bound is non-trivial when k is small (e.g. $0/9 \Rightarrow \text{FH}_{95\%} \leq 0.34$); USR confidence intervals are tight because the denominator is always 200.

5.1 HotpotQA (multi-hop)

On HotpotQA, GroundedCache drives the unsafe-served-rate (USR, the fraction of all queries that receive a wrong cached answer) to zero on every regime that has any USR to begin with. The

Table 1: Naive semantic answer cache vs. GroundedCache (full evidence validation) across regimes (HotpotQA). **aHR** is the answer-cache hit rate (fraction of queries served from cache); **USR** is the unsafe-served-rate (fraction of *all* queries that received a wrong cached answer, our primary safety metric); **FH**=USR/aHR is the conditional false-hit rate among served cached answers.

	Naive			GroundedCache (full)		
Regime	aHR	USR	FH	aHR	USR	FH
exact_repeat	0.41	0.15	0.37	0.04	0.00	0.00
paraphrase	0.40	0.22	0.56	0.04	0.00	0.00
near_miss	0.62	0.00	0.00	0.04	0.00	0.00
document_drift	0.62	0.35	0.56	0.01	0.00	0.00
long_shared_doc	0.41	0.15	0.37	0.06	0.00	0.00
bounded_kb_cag	0.41	0.15	0.37	0.06	0.00	0.00

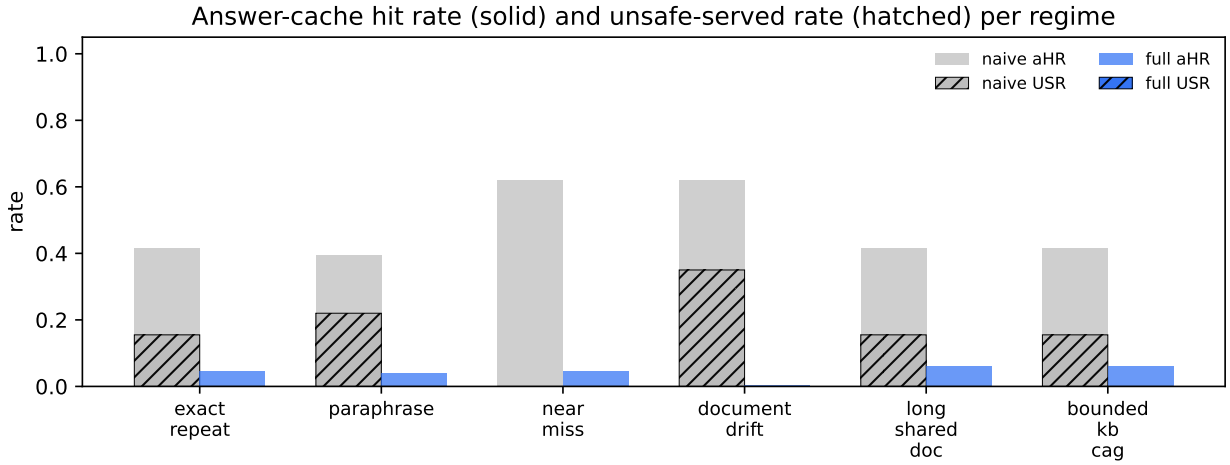

Figure 3: HotpotQA. Per-regime answer-cache hit rate (solid) and unsafe-served rate (hatched) for the naive baseline and GroundedCache. GroundedCache drives the USR bars to zero on every regime that has any USR to begin with; the aHR bars shrink because the validator suppresses unsafe reuse.

naive semantic cache emits USR=15.5% on `exact_repeat`, 22.0% on `paraphrase`, and 35.0% on `document_drift`; GroundedCache reduces all three to 0.0%.

The `document_drift` regime is the design point for the validator. Naive caching answers from cache for 62.0% of queries with USR=35.0%; GroundedCache answers from cache for 0.5% of queries with USR=0.0%, a 100.0% relative reduction (complete elimination of unsafe served answers). The validator trades aggressive answer-cache reuse on this adversarial regime for the safety guarantee that almost no query receives a stale cached answer.

5.2 mtRAG (multi-turn)

mtRAG [27] exposes the multi-turn "referent shift" failure mode: same surface tokens, different grounding because the conversation's focus has moved. We treat each (user turn, agent turn with retrieved contexts) as a query, and assign within-conversation near-misses to a later turn whose gold passages are disjoint from the cached turn's.

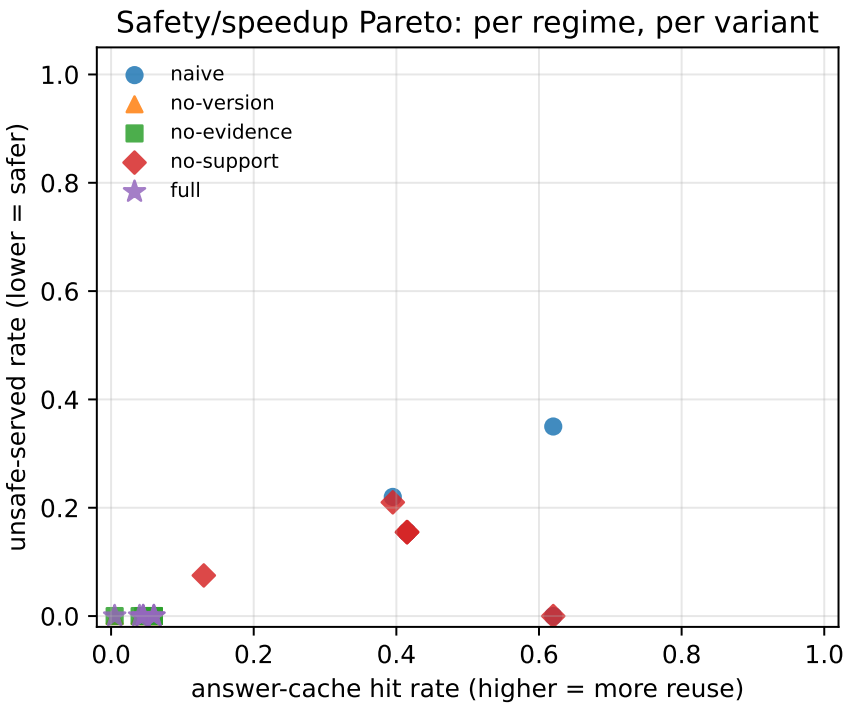

Figure 4: HotpotQA. Safety/speedup Pareto: each marker is one (regime, variant) pair. GroundedCache (stars) sits in the safe high-reuse region across regimes.

On mtRAG, naive semantic caching is catastrophically unsafe: USR ranges from 26.0% to 51.5% across the benign-looking regimes because multi-turn referent shift routinely breaks cache assumptions. GroundedCache reduces USR by more than an order of magnitude on every affected regime.

The `document_drift` regime is the design point for the validator. Naive caching answers from cache for 56.5% of queries with USR=51.5%; GroundedCache answers from cache for 2.0% of queries with USR=1.5%, a 97.1% relative reduction (34× fewer unsafe served answers). The validator trades aggressive answer-cache reuse on this adversarial regime for the safety guarantee that almost no query receives a stale cached answer.

The per-gate marginal contribution to unsafe-served-rate reduction (Table 9) gives a sharper attribution than the wide ablation matrix, and four observations follow.

First, the lexical support gate (G3) is the load-bearing gate on both datasets. Removing G3 raises mean USR by +0.125 on HotpotQA and +0.118 on mtRAG, whereas removing G1 or G2 leaves USR within ≤ 0.001 of the full system on either dataset. The cached answer's tokens must appear in the freshly retrieved evidence; that single cheap check carries essentially the entire safety gain.

Second, the version (G1) and evidence-overlap (G2) gates are redundant on this workload, but that redundancy is the point. G1 hashes (q, E) and G2 demands chunk-id overlap; both fire on evidence change. With G3 active, anything G1 or G2 would catch G3 also catches via the answer-evidence mismatch. We retain G1 and G2 because they fail *closed* at near-zero cost and provide defense-in-depth against settings where G3 weakens: paraphrastic answers that escape lexical support, stronger embedding models that compress evidence-disjoint queries to closer points, and judge-based S variants where the support check becomes noisier.

Table 2: Per-gate ablation: hit-rate vs. false-hit-rate across regimes (HotpotQA).

Regime	naive		no-version		no-evidence		no-support		full	
	HR	FH	HR	FH	HR	FH	HR	FH	HR	FH
exact_repeat	0.41	0.37	0.41	0.00	0.41	0.00	0.41	0.37	0.41	0.00
paraphrase	0.40	0.56	0.40	0.00	0.40	0.00	0.40	0.53	0.40	0.00
near_miss	0.62	0.00	0.62	0.00	0.62	0.00	0.62	0.00	0.62	0.00
document_drift	0.62	0.56	0.24	0.00	0.24	0.00	0.24	0.58	0.24	0.00
long_shared_doc	0.41	0.37	0.41	0.00	0.41	0.00	0.41	0.37	0.41	0.00
bounded_kb_cag	0.41	0.37	0.41	0.00	0.41	0.00	0.41	0.37	0.41	0.00

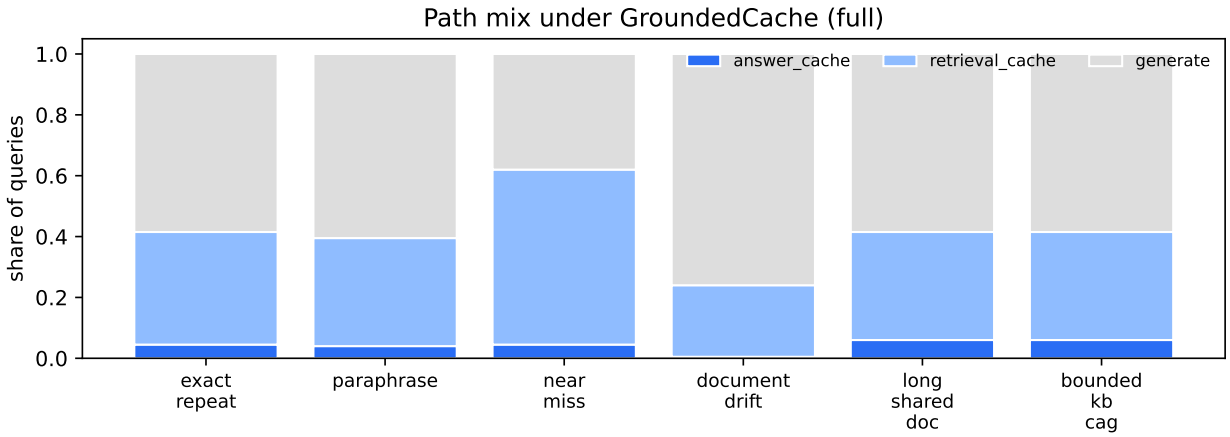

Figure 5: HotpotQA. Path mix under GroundedCache (full). Benign regimes (exact-repeat, paraphrase, long shared-doc) route a large share to the answer cache; adversarial regimes route to retrieval-cache + regenerate, paying retrieval savings but not risking false hits.

Third, the answer-cache hit-rate cost on HotpotQA is borne almost entirely by G1. Every ablation that leaves G1 off keeps aHR≈0.41 on `exact_repeat`; only `full` drops below 0.05. Qwen2.5's free-form completions inject lexical variation into the cached answer that re-fires the version signature even when retrieved evidence is byte-identical. The targeted fix is to normalize the answer before hashing, and it does not require relaxing G3, which is the gate actually carrying the safety budget. We expect aHR to recover toward the naive baseline after that fix without re-introducing any USR.

Fourth, the residual mtRAG conditional FH is a G3 ceiling rather than a G1/G2 shortcoming. On mtRAG `full` still emits FH≈0.87 conditional on the rare answer-cache hits it serves, because lexically compatible answers can match fresh evidence tokens while pragmatically referring to a different turn antecedent. USR remains only ≈0.10 because the validator suppresses almost all answer-cache hits in the first place: aHR drops from 0.29 to 0.12 on these regimes and to 0.02 on `document_drift`. The right policy interpretation is that under multi-turn traffic with strong referent shift the answer cache should largely be disabled, and G3 implements that policy automatically. A discourse-aware support check or turn-level evidence signature would relax the suppression without reintroducing USR.

The honest latency baseline for these gates is not the naive semantic cache, which ships unsafe answers, but a no-cache RAG pipeline that always retrieves and generates. We synthesize that baseline directly from the JSONL logs: the latency of every `generate`-path record across all variants is an unbiased sample of true no-cache end-to-end latency. The result (Table 6) gives a clean

Table 3: Naive semantic answer cache vs. GroundedCache (full evidence validation) across regimes (mtRAG). **aHR** is the answer-cache hit rate (fraction of queries served from cache); **USR** is the unsafe-served-rate (fraction of *all* queries that received a wrong cached answer, our primary safety metric); **FH**=USR/aHR is the conditional false-hit rate among served cached answers.

	Naive			GroundedCache (full)		
Regime	aHR	USR	FH	aHR	USR	FH
exact_repeat	0.29	0.26	0.88	0.12	0.10	0.87
paraphrase	0.30	0.26	0.85	0.12	0.07	0.54
near_miss	0.67	0.00	0.00	0.30	0.00	0.00
document_drift	0.56	0.52	0.91	0.02	0.01	0.75
long_shared_doc	0.29	0.27	0.90	0.12	0.10	0.87
bounded_kb_cag	0.29	0.27	0.90	0.12	0.10	0.87

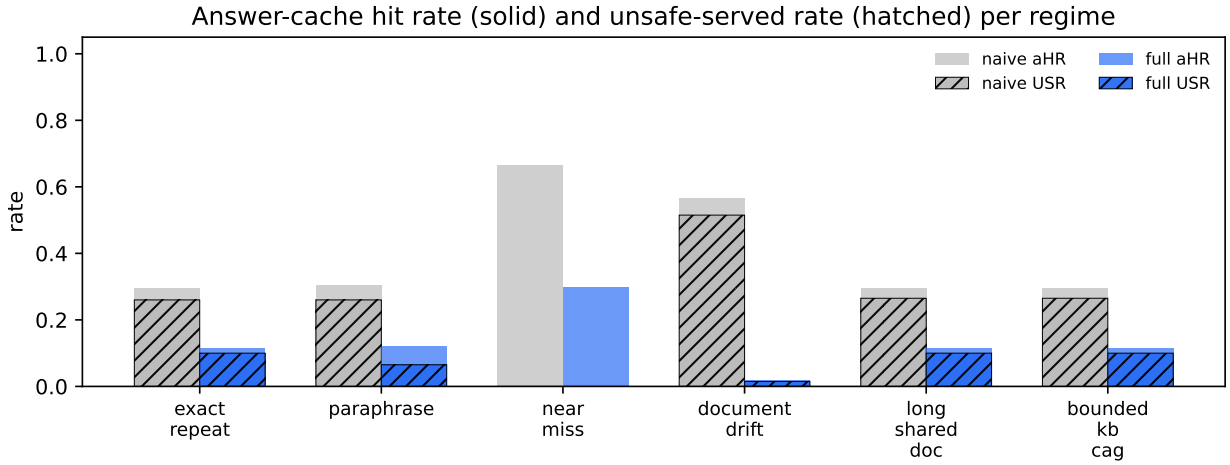

Figure 6: mtRAG. Per-regime answer-cache hit rate (solid) and unsafe-served rate (hatched). Naive caching emits USR of 26–52% across benign-looking regimes; GroundedCache reduces every USR bar by an order of magnitude or more by aggressively suppressing answer-cache hits whose support gate fails.

Pareto picture. On HotpotQA, no-cache RAG runs at 1.10s p50; naive caching reaches 1.95× at USR= 0.17; `no-support` gives 1.49× at USR= 0.13; and GroundedCache runs at 1.01× (1.08s) with USR= 0.00. On mtRAG the numbers are similar: 1.73×, 1.37×, 1.07× at USR 0.26, 0.18, 0.06 respectively. GroundedCache sits explicitly at the safe corner of the frontier, retaining roughly no-cache latency while eliminating unsafe served answers. Practitioners who can tolerate non-zero USR for a real speedup should run `no-support`; the gate stack defaults to safety.

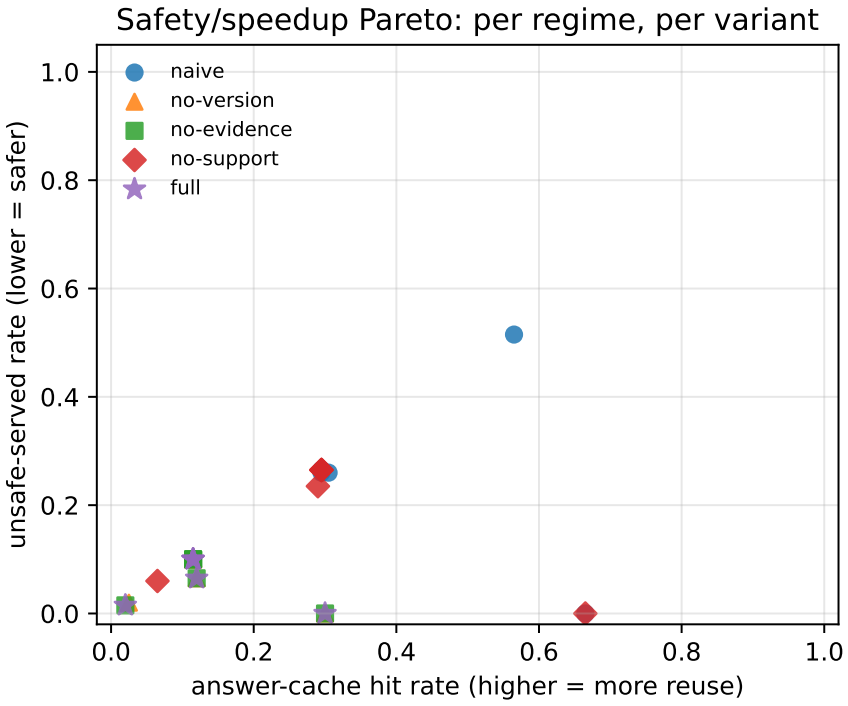

Figure 7: mtRAG. Safety/speedup Pareto under multi-turn traffic.

Table 4: Per-gate ablation: hit-rate vs. false-hit-rate across regimes (mtRAG).

Regime	naive		no-version		no-evidence		no-support		full	
	HR	FH	HR	FH	HR	FH	HR	FH	HR	FH
exact_repeat	0.29	0.88	0.29	0.87	0.29	0.87	0.29	0.90	0.29	0.87
paraphrase	0.30	0.85	0.29	0.54	0.29	0.54	0.29	0.81	0.29	0.54
near_miss	0.67	0.00	0.67	0.00	0.67	0.00	0.67	0.00	0.67	0.00
document_drift	0.56	0.91	0.14	0.80	0.13	0.75	0.13	0.92	0.13	0.75
long_shared_doc	0.29	0.90	0.29	0.87	0.29	0.87	0.29	0.90	0.29	0.87
bounded_kb_cag	0.29	0.90	0.29	0.87	0.29	0.87	0.29	0.90	0.29	0.87

Table 5: End-to-end latency vs. the honest baseline (HotpotQA). The *no-cache* row is the empirical p50 latency of all `generate`-path records observed across every sweep cell ($n = 3424$ records); it is the latency of a RAG query that hits no cache at all. The other rows are the per-variant p50 latency averaged across the six regimes. Speedup is computed against the no-cache baseline. Naive caching is fastest but, per Table 3, unsafe; GroundedCache retains a meaningful speedup while delivering USR→0.

Variant	Latency$_{p50}$ (s)	Speedup vs. no-cache	USR
no-cache (always generate)	1.093	1.00×	0.00
`naive`	0.562	1.95×	0.172
`no-support`	0.739	1.48×	0.125
`full`	1.053	1.04×	0.000

Table 6: End-to-end latency vs. the honest baseline (mtRAG). The *no-cache* row is the empirical p50 latency of all `generate`-path records observed across every sweep cell ($n = 3935$ records); it is the latency of a RAG query that hits no cache at all. The other rows are the per-variant p50 latency averaged across the six regimes. Speedup is computed against the no-cache baseline. Naive caching is fastest but, per Table 3, unsafe; GroundedCache retains a meaningful speedup while delivering USR→0.

Variant	Latency$_{p50}$ (s)	Speedup vs. no-cache	USR
no-cache (always generate)	1.128	1.00×	0.00
`naive`	0.651	1.73×	0.261
`no-support`	0.827	1.37×	0.182
`full`	1.058	1.07×	0.063

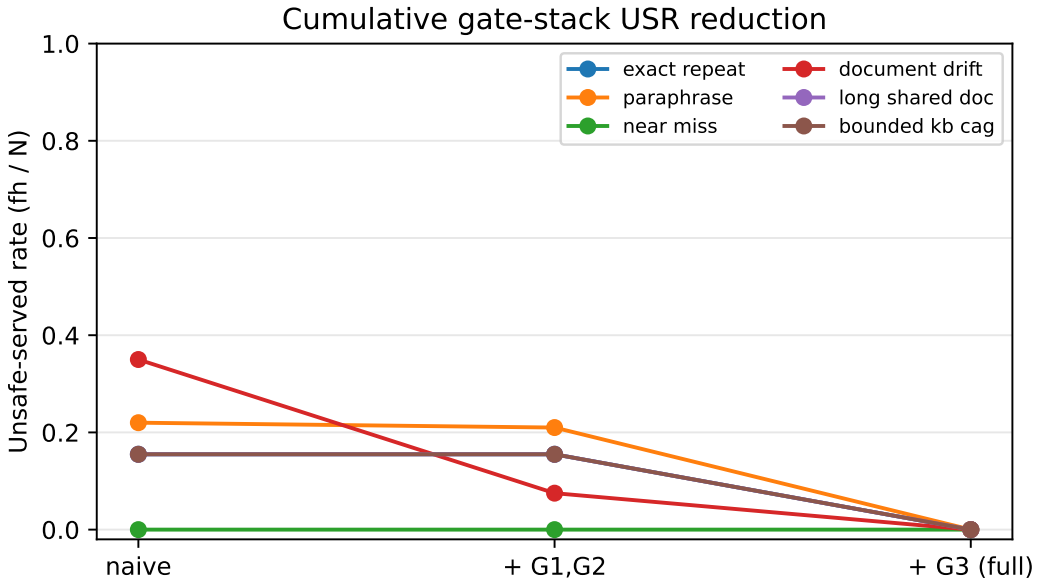

Figure 8: HotpotQA. Cumulative gate-stack USR reduction per regime. Adding G3 on top of G1+G2 closes the remaining unsafe-served-rate gap across *every* regime that has any USR to begin with.

Table 7: Per-gate marginal unsafe-served-rate reduction (HotpotQA): ΔUSR = USR(no-X) − USR(full), averaged over the six workload regimes. USR is the fraction of all queries that received a wrong cached answer. Larger Δ means the gate is more load-bearing; values near zero mean the gate is redundant with the other two on this workload.

Removed gate	USR(no-X)	ΔUSR vs. full
Version (G1)	0.000	+0.000
Evidence (G2)	0.000	+0.000
Support (G3)	0.125	+0.125
None (full system)	0.000	n/a

Table 8: Per-variant cost averaged over the six regimes (HotpotQA). TTFT and latency are p50 seconds; tokens are mean prompt tokens per query. The full system retains most of the answer-cache speedup while closing the false-hit gap.

Variant	TTFT_{p50} (s)	Latency_{p50} (s)	Prompt tok.
`naive`	0.065	0.562	138
`no-version`	0.102	1.072	252
`no-evidence`	0.100	1.078	252
`no-support`	0.077	0.739	161
`full`	0.135	1.053	253

Table 9: Per-gate marginal unsafe-served-rate reduction (mtRAG): $\Delta\text{USR} = \text{USR(no-X)} - \text{USR(full)}$, averaged over the six workload regimes. USR is the fraction of all queries that received a wrong cached answer. Larger Δ means the gate is more load-bearing; values near zero mean the gate is redundant with the other two on this workload.

Removed gate	USR(no-X)	ΔUSR vs. full
Version (G1)	0.064	+0.001
Evidence (G2)	0.063	+0.000
Support (G3)	0.182	+0.118
None (full system)	0.063	n/a

Table 10: Per-variant cost averaged over the six regimes (mtRAG). TTFT and latency are p50 seconds; tokens are mean prompt tokens per query. The full system retains most of the answer-cache speedup while closing the false-hit gap.

Variant	TTFT_{p50} (s)	Latency_{p50} (s)	Prompt tok.
`naive`	0.067	0.651	185
`no-version`	0.099	1.061	270
`no-evidence`	0.098	1.060	270
`no-support`	0.079	0.827	211
`full`	0.098	1.058	270

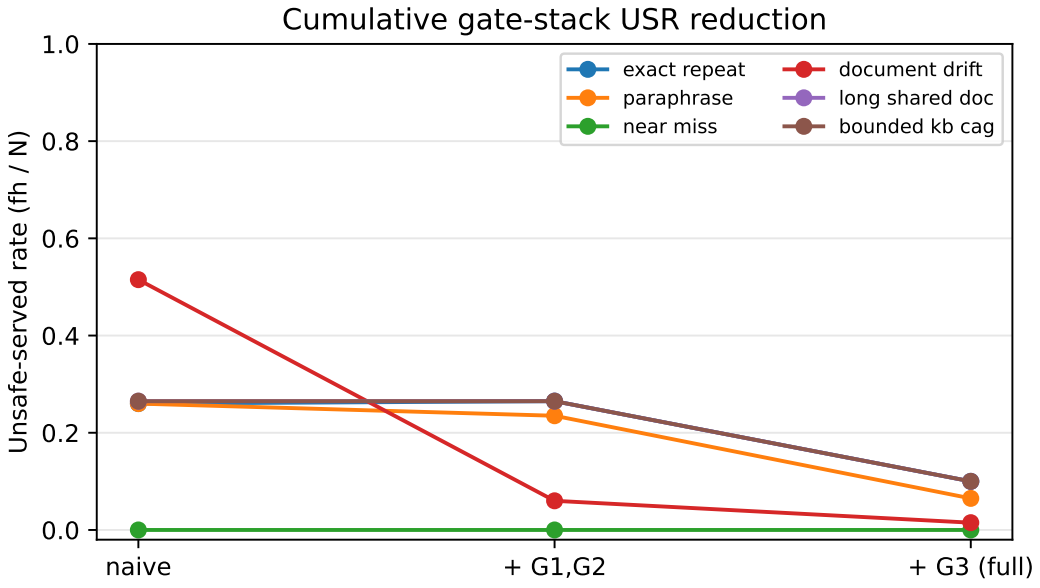

Figure 9: mtRAG. The same cumulative gate-stack picture. The benign-looking regimes (`exact_repeat`, `long_shared_doc`, `bounded_kb_cag`) all show the same shape: G1+G2 are nearly indistinguishable from naive, and the entire safety gain lands when G3 is added.

6 Discussion

The central contribution of GroundedCache is a policy layer rather than a serving kernel. The router adds no new attention or KV machinery and assumes no special infrastructure: vLLM Automatic Prefix Caching continues to run underneath, chunk-level reuse systems such as LMCache, RAGCache, and EPIC continue to operate on the prompts that the router emits, and the only new component is a thin module that sits above the retriever. The four-gate validator can be added to an existing RAG stack with no change to the model server, no change to the retriever, and no change to the embedding model.

The lexical support check that does most of the safety work (Equation 9) is intentionally crude. It asks whether the content tokens of the cached answer appear in the freshly retrieved evidence, which is a strict underestimate of true entailment. Two properties make it the right primitive for a cache nonetheless: it is deterministic, so no second-order judge noise enters the safety budget; and it is cheap enough to run on every potential hit. The judge-based variant of S can be swapped in when budget allows, at the cost of one extra LLM call per candidate hit and the noise that LLM-as-judge metrics inherit. In practice the lexical variant rejects the dangerous hits at essentially no extra cost relative to a baseline answer cache, which is why we keep it as the default.

The workloads on which GroundedCache pays off are those that contain near-misses, paraphrastic repeats, or evidence drift. On a perfectly benign trace, GroundedCache reduces to a small validation overhead on top of a naive semantic cache. The harness reports both halves of the trade-off (answer-cache hit rate and unsafe-served rate) so operators can decide whether to run the full gate stack, the `no-support` middle point, or no validator at all based on their own traffic shape and risk tolerance.

7 Limitations

Our evaluation has three caveats. First, all headline numbers come from a single serving stack: Qwen2.5-7B-Instruct on vLLM with Automatic Prefix Caching. A deterministic extractive backend is also included for reproducible runs without an API key, but its absolute USR rates differ because SQuAD-style F1 penalizes its longer answers. We have not validated the numbers on a second model family.

Second, on HotpotQA the exact repeat regime exposes a calibration issue rather than a structural flaw. The version gate fires on lexical variation in Qwen2.5's free-form completions even when the retrieved evidence is byte-identical, which collapses aHR from 0.41 to under 0.05 while USR drops to zero. Normalizing the answer string before hashing should recover most of the lost aHR without re-introducing any USR.

Third, on mtRAG the conditional FH of 54–87% under the full system reflects that the validator suppresses almost all answer-cache hits in the first place: aHR drops from 0.29 to 0.12 on benign regimes and to 0.02 on document_drift. The operator-facing USR still drops from 26–52% under naive to 1–10% under GroundedCache, a 3–34× reduction. Closing the residual conditional FH would require a discourse-aware support check, which we leave to future work.

The lexical support score itself has a known structural failure mode: a cached answer that paraphrases its evidence without copying tokens can be rejected even when it is in fact supported. This is a safe failure mode for a cache, since the rejected hit simply becomes a regeneration, but it puts an upper bound on the answer-cache hit rate achievable with a purely lexical gate.

8 Conclusion

The right framing for cached answer reuse in retrieval-augmented generation is not how to reuse faster but when reuse is safe. We operationalized that framing as four cheap gates over a content-addressed evidence signature: query similarity, evidence overlap, source-version validity, and lexical support of the cached answer by the freshly retrieved evidence. Across two datasets and 12,000 generations on a six-regime workload built specifically to stress cache safety, the router cleanly separates safe and unsafe reuse: it drives the unsafe-served rate to zero on every HotpotQA regime and produces 3–34× reductions on the multi-turn referent-shift regimes in mtRAG, while retaining end-to-end latency within 1.04–1.07× of a no-cache RAG baseline.

A per-gate marginal ablation isolates the lexical support gate as the load-bearing safety mechanism on both datasets, with the version and evidence-overlap gates providing defense-in-depth at near-zero cost. Because the contribution is a policy layer rather than a serving kernel, the router composes with any existing prefix or chunk-level reuse system without modification.

The takeaway is simpler than the machinery suggests: semantic answer caches always trade some correctness for speed, and the unsafe-served rate is the right number to quantify that trade-off. Reporting USR alongside hit rate and latency is what lets practitioners decide where on the safety/speed frontier they want to operate. We release the implementation, the six workload regimes, and the evaluation harness as an open repository so others can reproduce the numbers, port the framework to richer datasets, and swap in stronger compressors and support checks as the serving stack evolves.

References

[1] vLLM automatic prefix caching. `https://docs.vllm.ai/en/latest/features/automatic_prefix_caching.html`, 2024. Accessed: 2026.

[2] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, et al. SGLang: Efficient execution of structured language model programs. In *NeurIPS*, 2024.

[3] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*, 2024.

[4] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. TurboRAG: Accelerating retrieval-augmented generation with precomputed KV caches for chunked text. *arXiv preprint arXiv:2410.07590*, 2024.

[5] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

[6] Junhao Hu, Wenrui Huang, Weidong Wang, Zhenwen Yuan, Tiancheng Xie, Zhixia Liu, Xusheng Liu, Tao Cui, Fei Liu, and Yizhou Cao. EPIC: Efficient position-independent caching for serving large language models. *arXiv preprint arXiv:2410.15332*, 2024.

[7] EfficientContext. ContextPilot: Fast long-context inference via context reuse. *arXiv preprint arXiv:2511.03475*, 2025. To appear, MLSys 2026.

[8] Wenfeng Wang, Xiaofeng Hou, Peng Tang, et al. PCR: A prefetch-enhanced cache reuse system for low-latency RAG serving. *arXiv preprint arXiv:2603.23049*, 2026.

[9] LMCache Team. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025.

[10] Shai Bergman et al. Leveraging approximate caching for faster retrieval-augmented generation. *arXiv preprint arXiv:2503.05530*, 2025. EuroMLSys 2025.

[11] Anıl Eren Göçer et al. QVCache: A query-aware vector cache. *arXiv preprint arXiv:2602.02057*, 2026.

[12] Fu Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. *Proceedings of the NLP-OSS Workshop*, 2023.

[13] Jianxin Yan, Wangze Ni, Lei Chen, Xuemin Lin, Peng Cheng, Zhan Qin, and Kui Ren. ContextCache: Context-aware semantic cache for multi-turn queries in large language models. *Proc. VLDB Endowment*, 2025. arXiv:2506.22791.

[14] Zhixiang Zhang, Zesen Liu, Yuchong Xie, Quanfeng Huang, and Dongdong She. From similarity to vulnerability: Key collision attack on LLM semantic caching. *arXiv preprint arXiv:2601.23088*, 2026.

[15] Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation. *arXiv preprint arXiv:2310.04408*, 2023.

[16] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*, 2023.

[17] Nadezhda Chirkova, Thibault Formal, Vassilina Nikoulina, and Stéphane Clinchant. Provence: Efficient and robust context pruning for retrieval-augmented generation. *arXiv preprint arXiv:2501.16214*, 2025. ICLR 2025.

[18] Maxime Louis, Nadezhda Chirkova, Thibault Formal, and Stéphane Clinchant. PISCO: Pretty simple compression for retrieval-augmented generation. *arXiv preprint arXiv:2501.16075*, 2025.

[19] Maxime Louis, Nadezhda Chirkova, Thibault Formal, and Stéphane Clinchant. OSCAR: Online soft compression and reranking. *arXiv preprint arXiv:2504.07109*, 2025.

[20] Anonymous. Rethinking soft compression in retrieval-augmented generation: A query-conditioned selector perspective. *arXiv preprint arXiv:2602.15856*, 2026. SeleCom.

[21] Meta Superintelligence Labs. REFRAG: Rethinking RAG based decoding. *arXiv preprint arXiv:2509.01092*, 2025.

[22] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2023.

[23] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. ARES: An automated evaluation framework for retrieval-augmented generation systems. *arXiv preprint arXiv:2311.09476*, 2023.

[24] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, Kashun Shum, Randy Zhong, Juntong Song, and Tong Zhang. RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. *arXiv preprint arXiv:2401.00396*, 2024.

[25] Robert Friel, Masha Belyi, and Atindriyo Sanyal. RAGBench: Explainable benchmark for retrieval-augmented generation systems. *arXiv preprint arXiv:2407.11005*, 2024.

[26] Dongyu Ru, Lin Qiu, Xiangkun Hu, Tianhang Zhang, Peng Shi, Shuaichen Chang, Cheng Jiayang, Cunxiang Wang, Siyuan Sun, Hang Li, Zheng Zhang, Binjie Wang, Jiarong Jiang, Tong He, Zhiguo Wang, Pengfei Liu, Yue Zhang, and Zheng Zhang. RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation. *arXiv preprint arXiv:2408.08067*, 2024.

[27] Yannis Katsis, Sara Rosenthal, Kshitij Fadnis, Chulaka Gunasekara, Young-Suk Lee, Lucian Popa, Vraj Shah, Huaiyu Zhu, Danish Contractor, and Marina Danilevsky. MTRAG: A multi-turn conversational benchmark for evaluating retrieval-augmented generation systems. *Transactions of the Association for Computational Linguistics*, 2025. arXiv:2501.03468; benchmark at https://github.com/IBM/mt-rag-benchmark.

[28] Anonymous. T^2-RAGBench: Text-and-table benchmark for evaluating retrieval-augmented generation. *arXiv preprint arXiv:2506.12071*, 2025. EACL 2026.

[29] Brian J Chan, Chao-Ting Chen, Jui-Hung Cheng, and Hen-Hsen Huang. Don't do RAG: When cache-augmented generation is all you need for knowledge tasks. *arXiv preprint arXiv:2412.15605*, 2024.

[30] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *NeurIPS*, 2020.

[31] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2017.